

Trace It — My Complete Learning Guide

This document is your personal knowledge base for the Trace It project.
Read it slowly. Revisit it regularly. Everything is explained from scratch.

TABLE OF CONTENTS

1. What I Built — Project Summary
 2. Everything I Learned
 - Node.js & Express
 - MongoDB & Mongoose
 - JWT Authentication & Cookies
 - Middleware
 - File Uploads — Multer & Cloudinary
 - CORS
 - Python Flask & AI Matching
 - Docker & Docker Compose
 3. Bugs I Faced & How I Fixed Them
 - Bug: Sudden Logout After Page Refresh
 4. Revision Schedule
-

1. What I Built — Project Summary

Trace It is a Lost & Found platform where users can:

- Register/Login
- Report items they lost or found (with images)
- Use **AI** to find the closest semantic match between a lost item and all found items

The project has **3 separate services** running together:

```
Browser (React)
  ↓
Node.js Backend (Express) ← handles users, items, auth
  ↓
Python Backend (Flask)    ← handles AI semantic matching
  ↓
MongoDB Atlas (Cloud DB) ← stores all data
```

2. Everything I Learned

2.1 Node.js & Express

🤖 What is Node.js?

Node.js lets you run JavaScript on your **server** (not just in the browser). Before Node.js, JavaScript only ran inside browsers. Node.js changed that.

🤖 What is Express?

Express is a small framework built on top of Node.js that makes it easy to:

- Create routes (like `/api/login`, `/api/items`)
- Handle incoming requests
- Send back responses

📁 How this project uses it

```
server.js          ← Entry point. Starts the server on PORT 3000
src/app.js         ← Registers all routes and middleware
src/routes/        ← Defines which URL goes to which controller
src/controllers/   ← Actual logic (create item, delete item, etc.)
src/models/        ← Describes data shape in MongoDB
src/middlewares/   ← Code that runs in between (auth check, etc.)
```

✅ Key Concepts

Routes — a route is a URL pattern + what to do when someone visits it.

```
js

// When someone sends GET request to /api/lost → call getItem function
router.get("/", lostItemController.getItem)
```

Controllers — the function that actually handles the request.

```
js

async function getItem(req, res) {
  const items = await lostItemModel.find()
  return res.status(200).json({ items })
}
```

req = what the user sent (body, params, cookies, headers) **res** = what you send back to the user

Async/Await — database calls take time. `async/await` makes Node.js wait for the result before moving on, instead of crashing or sending back nothing.

2.2 MongoDB & Mongoose

🤔 What is MongoDB?

MongoDB is a **NoSQL database**. Instead of tables and rows (like Excel), it stores data as **documents** that look like JavaScript objects (JSON).

Example of one "LostItem" document stored in MongoDB:

```
json

{
  "_id": "abc123",
  "title": "Black Wallet",
  "description": "A black leather bifold wallet",
  "location": "Near bus stop",
  "category": "Accessories",
  "status": "Open",
  "reportedBy": "userId123"
}
```

🤔 What is Mongoose?

Mongoose is a library that lets Node.js talk to MongoDB easily. It also lets you define a **Schema** — a template/blueprint that says what fields a document must have.

✅ Schema Example from this project

```
js
```

```
const lostItemSchema = new mongoose.Schema({
  title: { type: String, required: true },
  description: { type: String, required: true },
  status: { type: String, enum: ["Open", "Closed"], default: "Open" },
  reportedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }
})
```

- `required: true` → this field MUST be present
- `default: "Open"` → if not provided, automatically set to "Open"
- `enum: ["Open", "Closed"]` → only these two values are allowed
- `ref: 'User'` → this field links to a User document (like a foreign key in SQL)

✓ Common Mongoose Operations

```
js
Model.find()           // Get all documents
Model.findById(id)      // Get one document by ID
Model.create({...})     // Insert new document
Model.findByIdAndUpdate() // Update by ID
Model.findByIdAndDelete() // Delete by ID
```

✓ Filtering with Query Parameters

```
js
// GET /api/lost?q=wallet&category=Accessories
const { q, category } = req.query

let filter = {}
if (q) {
  filter.$or = [
    { title: { $regex: q, $options: 'i' } }, // search in title
    { description: { $regex: q, $options: 'i' } } // search in description
  ]
}
if (category) filter.category = category

const items = await lostItemModel.find(filter)
```

- `$regex` = search using a pattern (like CTRL+F)
- `$options: 'i'` = case-insensitive (so "Wallet" and "wallet" both match)
- `$or` = match either condition

2.3 JWT Authentication & Cookies

🤔 What is Authentication?

Authentication = proving who you are.

When you log in, the server needs to remember you for all future requests.

HTTP is **stateless** — the server forgets you after every request. So we need a way to "attach your identity" to every request.

🤔 What is JWT?

JWT = **JSON Web Token**

It's like a **digital ID card** that the server gives you when you log in. You carry it and show it on every request.

A JWT looks like this (3 parts separated by dots):

```
eyJhbGciOiJIUzI1NiJ9.eyJpZCI6IjEyMyJ9.abc123signature
  ↑ Header           ↑ Payload (data)   ↑ Signature
```

The **Payload** contains info like your user ID. The **Signature** ensures no one tampered with it.

🤔 What is an HTTP-only Cookie?

A cookie is a small piece of data the browser stores and automatically sends on every request to the same server.

HTTP-only means JavaScript cannot read it. This is important for security — if your JS gets hacked (XSS attack), the attacker cannot steal your token.

✅ How this project uses JWT + Cookies

On Login:

```
js
const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET_KEY, { expiresIn: "1h" })

res.cookie("token", token, {
  httpOnly: true,    // JS cannot read this
  sameSite: "none",  // Allow cross-site (needed for different origin frontend)
  secure: true,       // Only send over HTTPS
  maxAge: 60 * 60 * 1000 // 1 hour
})
```

On Protected Routes (checking the token):

```
js

const token = req.cookies.token // Cookie is automatically sent by browser
const decoded = jwt.verify(token, process.env.JWT_SECRET_KEY)
req.user = decoded // Attach user info to the request
```

✅ Why `sameSite: "none"` and `secure: true`?

Setting	What it means
<code>sameSite: "none"</code>	Allow the cookie to be sent from a different origin (e.g., Vercel frontend to Render backend)
<code>secure: true</code>	Only send over HTTPS (required when sameSite is "none")
<code>httpOnly: true</code>	JavaScript can't read this cookie (security)

⚠️ Without these settings, your cookie won't work when frontend and backend are on different domains.

2.4 Middleware

🤖 What is Middleware?

Middleware is a function that runs **between** the request arriving and your controller executing.

Think of it like security at an airport:

```
User Request → [Security Check (middleware)] → Controller
```

If security fails → stop and send error back.

If security passes → continue to the controller.

✅ The Auth Middleware in this project

```
js
```

```

async function protectRoute(req, res, next) {
  const token = req.cookies.token

  if (!token) {
    return res.status(400).json({ message: "Unauthorized" })
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET_KEY)
    req.user = decoded // Save user info for controllers to use
    next()              // ← IMPORTANT: tells Express to continue
  } catch (err) {
    return res.status(400).json({ message: "Unauthorized" })
  }
}

```

- If token is missing → return 400 error, stop.
- If token is invalid → return 400 error, stop.
- If token is valid → attach user to `req`, call `next()` to proceed.

✅ How it's used on routes

```

js
// No protection – anyone can see lost items
router.get("/", lostItemController.getItem)

// Protected – only logged-in users can create
router.post("/create", protectRoute, upload.single("image"), lostItemController.create)
//                               ↑ runs first      ↑ runs second      ↑ runs last

```

2.5 File Uploads — Multer & Cloudinary

🤖 What is Multer?

Multer is middleware that handles **multipart/form-data** (what browsers send when uploading files).

Without Multer, `req.body` would be empty when you upload a file.

🤖 What is Cloudinary?

Cloudinary is a cloud service that stores your images online. Instead of saving images on your server (which gets wiped on redeploy), you upload to Cloudinary and get back a URL like:

`https://res.cloudinary.com/yourname/image/upload/TraceIt/abc.jpg`

That URL is what you store in MongoDB.

✅ How this project does it

```
js

// Step 1: Multer keeps the image in memory (RAM), not disk
const storage = multer.memoryStorage()
const upload = multer({ storage })

// Step 2: Upload the buffer (raw image data) to Cloudinary
const uploadToCloudinary = (fileBuffer) => {
  return new Promise((resolve, reject) => {
    cloudinary.uploader.upload_stream(
      { folder: "TraceIt" },
      (error, result) => {
        if (error) reject(error)
        else resolve(result.secure_url) // ← returns the image URL
      }
    ).end(fileBuffer)
  })
}

// Step 3: Use the URL when saving to MongoDB
const imageUrl = await uploadToCloudinary(req.file.buffer)
await lostItemModel.create({ ..., image: imageUrl })
```

Flow:

```
Browser uploads image
  → Multer catches it, holds it in RAM (req.file.buffer)
  → uploadToCloudinary() streams it to Cloudinary
  → Cloudinary returns a URL
  → URL is saved in MongoDB
```

2.6 CORS

🤔 What is CORS?

CORS = **Cross-Origin Resource Sharing**

Browsers have a **security rule**: a webpage at `https://myapp.vercel.app` cannot talk to a server at `https://myapi.render.com` **unless** the server explicitly says it's allowed.

If CORS is not configured, the browser blocks the request and you see:

Access to fetch at 'https://myapi.render.com' from origin 'https://myapp.vercel.app' has been blocked by CORS policy

✅ How this project configures CORS

```
js
app.use(cors({
  origin: function (origin, callback) {
    const allowedOrigins = [
      "http://localhost:5173",
      "https://trace-it-nu.vercel.app/"
    ]
    if (!origin || allowedOrigins.includes(origin) || origin.endsWith(".vercel.a
      callback(null, true) // ✅ Allow
    } else {
      callback(new Error("Not allowed by CORS")) // ❌ Block
    }
  },
  credentials: true // ← REQUIRED to allow cookies cross-origin
}))
```

⚠️ `credentials: true` is **essential** when using cookies. Without it, the browser won't send or accept cookies from a different origin.

2.7 Python Flask & AI Matching

🤔 What is Flask?

Flask is a lightweight web framework for Python.

It creates a simple HTTP server that can receive requests and send responses.

This project uses Flask **only for the AI part** — Node.js calls it internally. Users never directly talk to Flask.

🤔 How does AI Semantic Matching work?

Normal search (keyword): matches only if the exact word exists. Semantic search: matches **meaning**, not just words.

Example:

- Lost item: "black leather wallet"
- Found item: "dark bifold purse"

Keyword search → no match (no shared words)

Semantic search → HIGH match (both mean the same thing)

✅ How it works technically

Step 1 — Text → Vector (Embedding)

A sentence transformer model (`all-MiniLM-L6-v2`) converts any sentence into a list of 384 numbers (called an **embedding** or **vector**).

Similar sentences produce similar vectors.

```
"black leather wallet" → [0.21, 0.87, -0.34, ..., 0.55] (384 numbers)
"dark bifold purse"   → [0.19, 0.85, -0.31, ..., 0.52] (very similar!)
```

Step 2 — Cosine Similarity

Compare two vectors to get a score between -1 and 1.

- Score close to **1.0** = very similar meaning
- Score close to **0.0** = unrelated

```
python

scores = util.cos_sim(lost_embedding, found_embeddings)[0]
```

Step 3 — Rank Results

Sort all found items by their score (highest first).

```
python

results = sorted(results, key=lambda x: x['score'], reverse=True)
```

✅ How Node.js calls Python

```
js

// Node.js sends lost item description + all found descriptions to Python
const response = await axios.post(process.env.PYTHON_API_URL, {
  lost: description,
  found: foundItem.map(item => item.description)
})

// Python returns ranked matches
return res.status(200).json({ matches: response.data.matches })
```

Node Controller

- POST `http://python-ai:5000/match` (internal Docker network)
- Python scores and ranks
- Returns sorted matches
- Node sends to browser

2.8 Docker & Docker Compose

🤔 What is Docker?

Docker packages your app + all its dependencies (Node, Python, npm packages, etc.) into a **container** — an isolated box that runs the same everywhere.

┆ "It works on my machine" problem → Docker solves this.

🤔 What is Docker Compose?

Docker Compose lets you define and run **multiple containers** together with a single command:

```
docker compose up
```

✅ This project's docker-compose.yml explained

yaml

```
services:
  python-ai:          # Service 1: Python Flask AI
    build: ./backend-python
    ports: ["5000:5000"] # host:container

  node-backend:       # Service 2: Node.js API
    build: ./backend-node
    ports: ["3000:3000"]
    depends_on: [python-ai] # Start python-ai first

  frontend:           # Service 3: React (served by Nginx)
    build: ./frontend
    ports: ["3001:80"]
    depends_on: [node-backend]

networks:
  traceit-net:        # All 3 services share this private network
```

Key insight: Inside Docker's network, services talk to each other by name:

- Node calls Python as: `http://python-ai:5000` (not localhost!)

- Why? Each container is its own "computer". `localhost` inside Node's container refers to Node's container only — not Python's container.

✅ Dockerfile basics (Node example)

dockerfile

```
FROM node:20-alpine      # Start with a Node.js image
WORKDIR /app             # Set working directory inside container
COPY package*.json ./    # Copy package.json first (for caching)
RUN npm ci --only=production # Install dependencies
COPY . .                 # Copy rest of code
EXPOSE 3000              # Tell Docker this app uses port 3000
CMD ["node", "server.js"] # Run this when container starts
```

3. Bugs I Faced & How I Fixed Them

3.1 Bug: Sudden Logout After Page Refresh

🐛 What was happening?

Every time the page was refreshed, the user got logged out — even though they had logged in just seconds ago.

😬 Why did this happen?

This is one of the most **common bugs** in React + JWT cookie auth.

Here's the root cause:

```
User logs in
  → Server sends JWT in HTTP-only cookie ✅ (saved in browser)
  → React stores user info in Context/State: { username, email } ✅

User refreshes the page
  → React state resets to null (memory is wiped on refresh) ❌
  → App checks: "Is user logged in?" → State says null → LOGGED OUT ❌
```

The cookie was still there in the browser. The JWT was still valid.

But React didn't know because it only stored auth state in memory.

The browser only sends the cookie — it doesn't automatically tell React "hey, this cookie exists, please log in the user."

✅ How it was fixed

The fix: on every page load (app mount), call `GET /api/auth/me` to check if a valid cookie exists and re-hydrate the auth state.

In `AuthContext.jsx`:

```
jsx

const [user, setUser] = useState(null)
const [loading, setLoading] = useState(true) // important!

useEffect(() => {
  // This runs once when the app first loads
  const checkAuth = async () => {
    try {
      const res = await axios.get("/api/auth/me", { withCredentials: true })
      setUser(res.data.user) // ✅ Re-hydrate user from cookie
    } catch (err) {
      setUser(null) // No valid cookie → treat as logged out
    } finally {
      setLoading(false)
    }
  }

  checkAuth()
}, [])

// ⚠️ Show loading spinner while checking – prevents flash of "logged out" screen
if (loading) return <div>Loading...</div>
```

And on the backend, the `/api/auth/me` route:

```
js

async function checkAuth(req, res) {
  const user = await userModel.findById(req.user.id).select("-password")
  return res.status(200).json({ user })
}
```

Flow after fix:

```
App loads in browser
→ useEffect triggers checkAuth()
→ Browser sends request to /api/auth/me WITH the cookie (automatically)
→ Server reads cookie → verifies JWT → returns user data
→ React sets user state ✅
→ User is now "logged in" again without re-entering credentials
```

Lesson Learned

React state lives in memory and resets on refresh. Cookies persist in the browser across refreshes. You must always "sync" React state with the cookie on app load. The `loading` state is critical — without it, the app briefly shows the user as "logged out" before the check completes (causes screen flash).

4. Revision Schedule

Use this schedule to keep your knowledge fresh.

Each session should take 15–30 minutes — re-read the relevant section and mentally walk through the code.

Session 1 — Day 3 from today

Focus: Node.js, Express, Routes, Controllers

Questions to answer from memory:

- What does `req` and `res` contain?
 - What is the difference between a route and a controller?
 - What does `async/await` do?
-

Session 2 — Day 7 from today

Focus: MongoDB, Mongoose, Schemas, Queries

Questions to answer from memory:

- What is the difference between MongoDB and SQL?
 - What does `$regex`, `$or`, and `$options: 'i'` do?
 - What does `ref: 'User'` mean in a Schema?
-

Session 3 — Day 14 from today

Focus: JWT, Cookies, Auth Middleware

Questions to answer from memory:

- Why do we use HTTP-only cookies instead of `localStorage`?
- What does `jwt.sign()` do? What does `jwt.verify()` do?

- What is the role of `next()` in middleware?
 - Why did the logout-on-refresh bug happen? How was it fixed?
-

📅 Session 4 — Day 21 from today

Focus: File Uploads, CORS, Docker

Questions to answer from memory:

- What does Multer do? Why do we use `memoryStorage`?
 - Why is `credentials: true` needed in CORS?
 - Why can't Node use `localhost` to talk to Python in Docker?
 - What does `depends_on` do in docker-compose?
-

📅 Session 5 — Day 30 from today

Focus: Full project walkthrough

Do this:

1. Open the project folder
 2. Start from `server.js` and trace a full request: `POST /api/auth/login` → all the way to the JWT cookie response
 3. Then trace: `GET /api/lost/match/:id` → all the way to Python and back
-

📅 Session 6 — Day 45 from today

Focus: Re-read this entire document

By now you should be able to explain every section to someone else.

If anything feels unclear → that's the area to review more deeply.

📅 Session 7 — Day 60 from today

Focus: Build something new using these concepts

Challenge: Add a new feature to Trace It or start a new small project using Express + MongoDB + JWT. The best way to remember is to re-apply.

★ Quick Reference Card

Concept	One-line summary
Express Route	A URL pattern + handler function
Mongoose Schema	Blueprint for what a MongoDB document looks like
JWT	Signed token containing user identity
HTTP-only Cookie	Stores JWT securely; JS cannot read it
Middleware	Code that runs between request and controller
Multer	Handles file uploads in Node.js
Cloudinary	Cloud storage for images, returns a URL
CORS	Browser security rule; server must allow cross-origin requests
Semantic Matching	AI finds similar meaning, not just matching words
Docker Container	Isolated box with app + all its dependencies
Docker Compose	Runs multiple containers together
<code>next()</code>	Tells Express to move to the next middleware/route
<code>req.user</code>	User info attached by auth middleware
<code>withCredentials: true</code>	Tells Axios to send cookies cross-origin

Document created: March 2026 — Trace It project by Mumuksh Jain